

2D Effect Implementation Tool

Maci Boone (Texas A&M University), Jason Chen (University of Pennsylvania), William Joel (Marist University)

Introduction

Modern animation pipelines from studios such as Pixar and Sony have preferred to use 3D animation software over hand-drawn 2D animation due to the efficiency and higher technical capabilities of computers. In recent films such as *Spider-Man: Into the Spider-Verse*, these handmade techniques have been reintroduced in production pipelines using 3D animation software to blend computer rendered animation with handmade 2D effects (Dimian et al.). Achieving this look, however, requires 2D effects to be composited post-render, a workflow that makes iteration slow and expensive. This project investigated the development of tools within 3D software applications, such as Blender, to streamline the integration of 2D hand-drawn effects into 3D rendering pipelines.

Development

The project began by focusing on how 2D effects could be displayed on a canvas (plane) in a 3D scene that is integrated within the animation timeline. Blender was selected because of how commonly it is used by smaller creators and the extensive control it provides through Python scripts.

In initial stages, the objective was to assign a 2D canvas to a target model such that hand-drawn effects can be drawn on a canvas in a 3D scene. We chose to use Blender's plane object to act as our canvas. A script was created that instantiates a canvas at the origin of the selected object, scaling the canvas in proportion to the object. The canvas is then constrained to the position of the target model by copying its position, so that it is in sync with the object. Next, an additional constraint using the "TRACK_TO" function was applied to the orientation of the canvas which ensured it always faced the camera placed in the scene (See Figure 1). This allowed the canvas to remain the same scale as the target model moved or rotated. To trigger the use of the tool, a button was created in Blender's dopesheet, which assigns the canvas to the target model when clicked (Blender Stack Exchange).

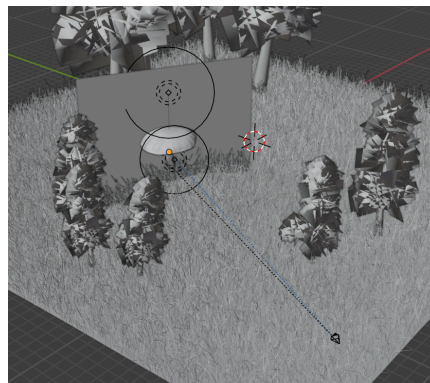


Figure 1: Instantiating a plane and constraining its position and rotation to the target model

```
def billboard():
    bpy.ops.mesh.primitive_plane_add()
    plane = bpy.context.object

    plane.scale = (currentObj.dimensions.x * borderMultiplier, currentObj.dimensions.y *
borderMultiplier, 0)

    copyLoc = plane.constraints.new(type='COPY_LOCATION')
    copyLoc.target = currentObj

    trackTo = plane.constraints.new(type='TRACK_TO')
    trackTo.target = bpy.context.scene.camera
```

Figure 2: Code for the plane to follow the model while facing the camera

The next task was to project a 2D effect onto the canvas. For maximum flexibility, emphasis was placed on how to assign hand-drawn animations with transparent backgrounds, since animators would want objects behind the animation to be seen. The first issue encountered was that while animations could be in any format, Blender itself only accepted image sequences. Since transparency needed to be considered, only sequences from GIFs or MOVs could be used. GIFs were the primary focus of the project due to their wide usage. To implement the GIF, the GIF needs to be imported as an image sequence into Blender and plugged into a material in the base color and alpha channels. To assign the material to the canvas, a user interface panel was created in the 3D viewport. This interface allows the user to select their desired material and automatically apply the material to the 2D canvas.

To provide greater control within Blender's timeline, the UI panel contains parameters to set the start and end frames of the 2D animation, along with an additional parameter to enable looping of the animation. The canvas appears and the animation begins playing at the start frame. If animation is supposed to be looping, then it loops until the end frame. Otherwise, the animation stops playing and the canvas is hidden at the end frame (See Figure 3). These actions are accomplished using the "hide_viewport" and "hide_render" operations, which were chosen because they were the easiest way to hide a canvas in the render while being able to see the animation in effect while working in the scene.

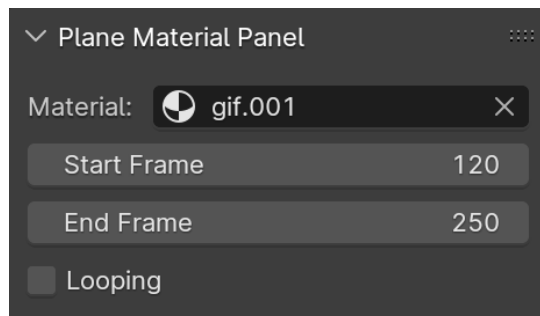


Figure 3: Creating an interactive UI panel in the Blender viewport

Discussion

Applying 2D effects in a 3D environment presents several challenges. In many production pipelines, 2D effects are implemented during compositing workflows long after the animations have undergone lengthy rendering times. For example, from *Spider-Man: Into the Spider-Verse*, they employed a 2D artist to insert spritesheets into 3D scenes in Houdini in order to see the effects play alongside 3D characters, and often moved between 3D renders and 2D compositing. This workflow is complicated to set up and significantly restricts the ability of artists to make small but necessary adjustments. If the compositor discovers that a 2D effect does not fit properly within the animation, either the effect or the animation or both must be revised, any of which requires a significant amount of time and resources (O'Hailey).

The tool developed in this study eliminates this lengthy process by bringing 2D effects directly into the 3D animation software, allowing them to be rendered alongside the animation itself. The concept functions similarly to Blender's Grease Pencil, where 2D line drawing can be drawn and rendered within the same environment as the 3D animation. Our tool, however, offers more useful functionality since it allows all types of 2D effects to be brought into a 3D scene, instead of having to draw them within Blender. Then, in a few steps, it allows the animator to get feedback about their 2D effect real-time, and the ability to see them interact with the 3D scene lets them make edits faster.

While the current prototype successfully integrates 2D effects into the 3D viewport, there are still limitations in its functionality. Blender does not allow GIF files to be directly imported as a material, so a GIF must be split into an image sequence before it can be successfully implemented. Thus, next steps in the project involve implementing a system that takes in hand-drawn effects in any form such as images or videos, then automatically splits videos into image sequences. Additionally, the current system assumes only one main camera is in the scene so the canvas can be correctly oriented towards the viewer. Also, animators may want to have the canvas oriented in a different way away from the camera. Animators might also wish to have more freedom when positioning the 2D effect relative to the target model in 3D space, or possibly having multiple 2D effects layered in a specific order on one target model. Addressing these limitations will be the next steps in readying this tool for complex production environments.

Conclusion

Overall, this project successfully produced a working prototype that achieves the intended goal of improving the efficiency of adding 2D effects into 3D animation pipelines. Going forward, there are several opportunities for improvement. With regards to using Blender's materials, there will be further exploration into streamlining the input of a 2D effect without having to use image sequences. Additional improvements include altering where the canvas is summoned with respect to the target object, along with orienting the canvas with multiple cameras in the scene. Further development will enable greater strides in efficiency in hybrid animation workflows. A demonstration of the tool in use within Blender, along with a final rendered output, is provided in the supplemental video.



Figure 4: Screenshot from rendered animation using the 2D effects tool

Acknowledgments

The authors would like to acknowledge the following students who have worked on this project over the years: Esosa Asaolu (Marist University), Noah Cho (University of British Columbia), Zhifang Han (Bournemouth University), Roy Hridiza (Rochester Institute of Technology), Sing Chun Lee (Bucknell University), Elina Lescinsca (Bournemouth University), Isaac Nuttall (Bournemouth University), Ken Saito (Keio University), Kim Siyoung (Georgia Institute of Technology), and Eric Yee (University of British Columbia). Also the Computer Graphics & Animation Research Projects program of which this project was a part of.

References

- Blender10der. "Running a Script with the Timeline." *Blender Artists Community*, 12 Mar. 2020, blenderartists.org/t/running-a-script-with-the-timeline/1212354. Accessed 15 Nov. 2025.
- Dimian, Danny, et al. "Swing into Another Dimension: The Making of *Spider-Man: Into the Spider-Verse*." *ACM SIGGRAPH 2019 Production Sessions*, ACM, 28 July 2019, doi:10.1145/3292423.3313758.
- "Header(BPY_STRUCT)." *Header(Bpy_struct) - Blender Python API*, docs.blender.org/api/current/bpy.types.Header.html. Accessed 15 Nov. 2025.
- O'Hailey, Tina. *Hybrid Animation: Integrating 2D and 3D Assets*. Taylor and Francis, 2012.
- "(Python) Add Ui Button to the Grease Pencil Dopesheet Header Using Python." *Blender Stack Exchange*, <https://stackoverflow.com/#organization>, 15 Feb. 2023, blender.stackexchange.com/questions/283342/python-add-ui-button-to-the-grease-pencil-dopesheet-header-using-python. Accessed 15 Nov. 2025.